Enhanced Semantic Segmentation of LiDAR Point Clouds Using Projection-Based Deep Learning Networks

Qihui Li, *Graduate Student Member, IEEE*, Qiliang Du, Lianfang Tian, *Senior Member, IEEE*, Wenzhi Liao, *Senior Member, IEEE*, and Guoyu Lu, *Member, IEEE*

***Abstract*—Light detection and ranging (LiDAR) point cloud semantic segmentation has emerged as a fundamental technique for enabling intelligent perception in autonomous driving, robotics, and geospatial analysis. Point cloud segmentation methods are typically categorized into three types: point-based, voxel-based, and projection-based techniques. While point-based and voxel-based methods offer robust feature extraction, they face challenges related to computational efficiency and the handling of large-scale point clouds. Projection-based methods, on the other hand, project 3-D point clouds into 2-D representations, enabling the application of established 2-D convolutional neural networks (CNNs) for segmentation tasks. Despite their advantages in efficiency, projection-based methods often suffer from the loss of spatial precision, leading to suboptimal segmentation performance, especially in complex and cluttered environments. In this article, we propose a novel projection-based approach for semantic segmentation that addresses the limitations of existing methods. Our approach introduces a multiscale feature embedding (MSFE) module to enhance feature extraction from the projected range images, combined with a multifeature fusion module (MFFM) to integrate features at multiple scales. We further improve segmentation accuracy for challenging objects, such as pedestrians, traffic signs, and occluded structures, by utilizing a dual segmentation head (DSH). Our experiments on the SemanticPOSS and SemanticKITTI datasets show significant improvements over existing methods, achieving a mean intersection over union (mIoU) of 53.6% on SemanticPOSS and 67.8% on SemanticKITTI. Notably, we achieve high performance on small and occluded objects, like trashcans (55.9% mIoU) and fences (49.5% mIoU), demonstrating the effectiveness of our approach for real-world applications like autonomous driving.**

***Index Terms*—Autonomous driving, light detection and ranging (LiDAR), point cloud, range sensing, semantic segmentation.**

Received 26 May 2025; revised 18 September 2025 and 16 October 2025; accepted 29 October 2025. Date of publication 3 November 2025; date of current version 7 November 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB4704900, in part by the National Natural Science Foundation of China under Grant 62476094, and in part by Zhuhai Industry-University-Institute Cooperation Project under Grant 2220004002460. *(Corresponding authors: Qiliang Du; Lianfang Tian.)*

Qihui Li, Qiliang Du, and Lianfang Tian are with the School of Automation Science and Engineering, South China University of Technology, Guangzhou 510641, China (e-mail: qihuili88@gmail.com; qldu@scut.edu.cn; chlftian@scut.edu.cn).

Wenzhi Liao is with the Sustainable Materials Management Unit, Flemish Institute for Technological Research (VITO), 2400 Mol, Belgium, and also with the Department of TELIN, Ghent University, 9000 Ghent, Belgium (e-mail: wenzhi.liao@vito.be).

Guoyu Lu is with the Intelligent Vision and Sensing Laboratory, Binghamton University, Binghamton, NY 13902 USA (e-mail: guoyulu62@gmail.com).

Digital Object Identifier 10.1109/TGRS.2025.3627917

I. Introduction

As 3-D sensors become increasingly prevalent, point cloud semantic segmentation has emerged as a vital research area with critical applications in autonomous driving [1], [2], large-scale geospatial mapping [7], [8], [9], and mobile robotic navigation. The increasing demand for real-time, accurate segmentation in these domains has spurred the development of numerous point cloud segmentation methods. Broadly, as shown in Fig. 1, these methods can be categorized into three types: point-based [22], [23], [26], [29], voxel-based [31] and [33], and projection-based [5], [19], [44], [45], [57], [61], [62] approaches.

Point-based methods [3], [4], [6], [22], [23] directly process raw point cloud data without converting it to a grid. Methods like BAAF [29] use attention mechanisms to focus on important points and reduce noise. RandLA-Net [26] applies random sampling and local aggregation to efficiently process large point clouds while maintaining accuracy. 3D-VQAF [12] introduces a hyperspectral point cloud projection (HPProj) model to combine hyperspectral and light detection and ranging (LiDAR) data for better multimodal fusion. DWS-KP-FCNN [13] uses dynamic weighted sampling (DWS) with KPConv to handle class imbalance by focusing more on underrepresented classes. While these methods excel at segmenting based on local features, they face scalability issues with large-scale point clouds, commonly found in autonomous driving, due to high point density and computational challenges. Voxel-based methods, on the other hand, convert point clouds into regular 3-D grids (voxels), which can then be processed using conventional convolutional neural networks (CNNs). While voxel-based methods, like MinkowskiNet [31] and SPVNAS [33], offer computational efficiency, they are hindered by increased memory consumption and computational overhead, especially when dealing with high-density point clouds that are common in practical scenarios.

Projection-based methods [44], [57], [61], [62] convert 3-D point clouds into 2-D projections (e.g., range images or depth maps) and use 2-D CNNs for segmentation. These methods benefit from the efficiency of 2-D CNNs, which are better suited for handling large datasets. However, a key limitation is the loss of spatial information during projection, which can reduce segmentation accuracy, especially for complex or occluded objects [16], [17]. While projection-based methods have made significant progress, existing architec-

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

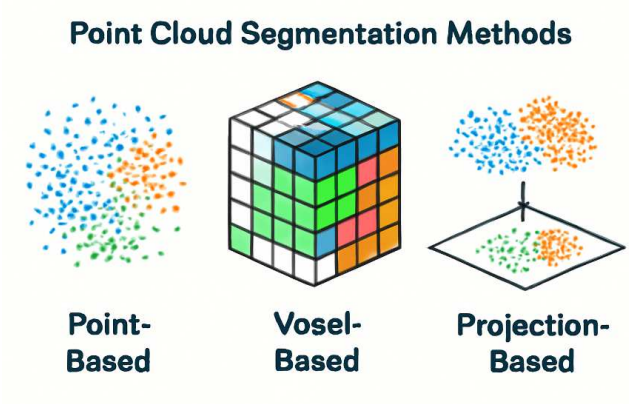


Fig. 1. Common point cloud processing methods. These include point-based methods, voxel-based methods, and projection-based methods. Notably, projection-based methods demonstrate higher efficiency when handling large-scale point cloud data.

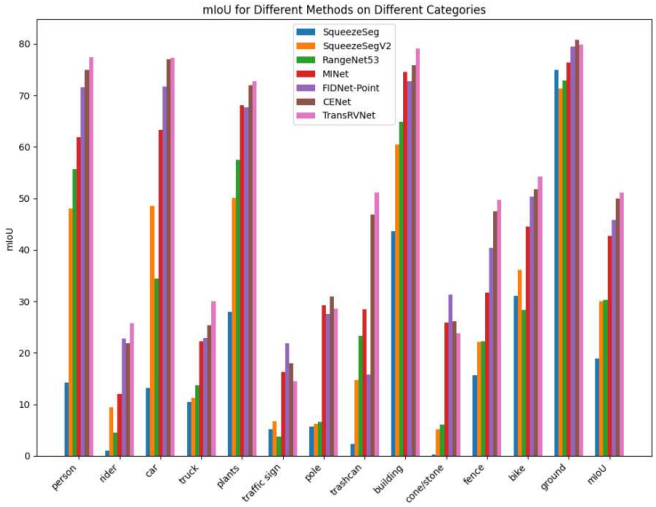


Fig. 2. Segmentation results of several existing methods using a bar chart. It is evident that the categories "rider," "truck," "traffic sign," "pole," and "cone/stone" exhibit significantly low segmentation accuracy, indicating that they are challenging categories. The segmentation results for these categories are approximately only half as accurate compared with other categories.

tures still exhibit a variety of limitations that hinder their overall effectiveness. This article analyzes the components of current projection-based structures and proposes specific improvements to address their shortcomings, particularly in the context of segmenting difficult-to-detect and occluded objects.

One major limitation of current projection networks, such as SqueezeSeg [14], SqueezeSegV2 [15], and CENet [19], is that their embedding layers use a single linear layer to project the image into higher dimensions before passing the features to the backbone. This simplification, while computationally efficient, often fails to capture the complex relationships inherent in the projected image. To address this, we conduct ablation studies that demonstrate the crucial importance of the embedding design for the overall network performance. We propose the multiscale feature embedding (MSFE) module, which extracts multiscale features from the projected image. Through an innovative channel-mixing technique, we effectively fuse the multiscale features, allowing the network to better capture fine-grained information. Our embedding layer utilizes multiple MSFE modules, and with the addition of skip connections, we mitigate overfitting and significantly enhance the network's ability to capture relevant features. This method offers a simpler, yet more effective alternative to the complex backbone designs typically used in the field.

Next, we incorporate a multifeature fusion module (MFFM) to further improve feature extraction. By applying channel and spatial attention fusion, we pairwise combine multiple feature maps, fully integrating multilevel features. This enables the network to capture a richer set of features at different resolutions. In comparison to existing approaches [15], [19], [20], [38], which rely on skip connections for upsampling, our method achieves a more comprehensive and balanced feature fusion that boosts segmentation performance.

Finally, we introduce the dual segmentation head (DSH), which addresses a critical limitation in existing methods: imbalanced segmentation performance across different object categories. As shown in Fig. 2, existing networks [14], [15], [16], [17], [18], [19], [20] struggle with segmenting difficult objects, such as riders, trucks, traffic signs, poles, and cones/rocks. The segmentation accuracy for these objects is significantly lower than for other categories, indicating a substantial weakness in handling these challenging cases. To tackle this issue, we introduce an auxiliary branch in the DSH, specifically designed to focus on segmenting these difficult objects. The dual-head fusion mechanism significantly improves segmentation accuracy for challenging categories. In addition, we enhance the overall performance by incorporating features reconstructed by the backbone during the segmentation process. The overall process of our method is shown in Fig. 3. In summary, our contributions are as follows.

1) We design the MSFE, which enhances the network's ability to capture multiscale features from projected images, effectively improving the network's segmentation capacity.
2) We propose the MFFM to fully integrate multilevel upsampled features, thus effectively merging the backbone's multilevel features.
3) Finally, we introduce the DSH, with an auxiliary branch that focuses on segmenting challenging objects, significantly improving the segmentation performance for these difficult-to-segment categories.

II. Related Works

In this section, we will focus on introducing three primary methods for processing point clouds: point-based methods (see Section II-A), voxel-based methods (see Section II-B), and image-based methods (see Section II-C).

A. Point-Based Learning

Point-based learning methods process raw point cloud data directly without converting it into other representations, making them highly efficient for tasks involving unstructured 3-D data. PointNet [22] and its hierarchical extension PointNet++ [23] laid the foundation for pointwise classification. KPConv

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

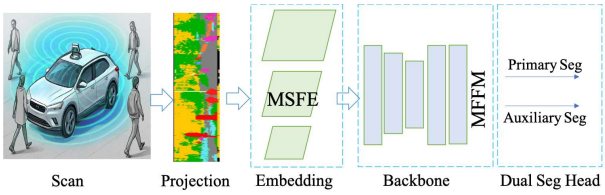


Fig. 3. Workflow of our method. After the point cloud is projected, feature extraction is performed using the MSFE, followed by multilevel feature extraction within the backbone. Subsequently, the MFFM aggregates the multilevel features, and the segmentation task is executed using the DSHs.

[28] and RandLA-Net [26] enhanced local geometric feature extraction and scalability to large scenes. BAAF [29] improves point cloud segmentation by leveraging bilateral structures and adaptive fusion techniques, effectively combining both geometric and semantic features. This leads to more precise semantic segmentation results. GFNet [50] integrates geometric learning and feature learning into an end-to-end framework. By leveraging global feature aggregation, it improves the performance of 3-D point cloud semantic segmentation, particularly by enhancing the capture of both geometric and semantic information. PTv2 [51] and PTv3 [52] leverage transformer-based architectures to process point clouds with long-range dependencies, providing a robust framework for capturing global context and improving segmentation accuracy, particularly in complex environments.

B. Voxel-Based Learning

Voxel-based methods discretize point clouds into 3-D grids or voxels, converting irregular point data into a regular structure that can be processed by conventional CNNs. MinkowskiNet [31] proposes the use of sparse convolutions for 3-D point cloud data, reducing computational costs while maintaining high performance in semantic segmentation tasks. This approach is highly efficient for large-scale 3-D data. SPVNAS [33] utilizes neural architecture search (NAS) to optimize voxel-based networks, focusing on improving performance while maintaining computational efficiency. Cylinder3D [34] is designed to leverage cylindrical coordinate systems for better handling of data with significant variations in scale and orientation, particularly for LiDAR point clouds in autonomous driving. OA-CNNs [53] introduce an omni-adaptive sparse CNN framework that utilizes octree structures to efficiently process sparse voxelized representations. By focusing on nonempty voxels, this method significantly reduces unnecessary computation and enhances performance in 3-D semantic segmentation tasks. PV-RCNN [55] combines point cloud processing with voxelization in a hybrid model that achieves improved performance on large-scale 3-D data, making it particularly effective for applications in robotics and autonomous driving.

C. Image-Based Learning

Image-based learning methods project 3-D point clouds onto 2-D planes, such as range images or depth maps, allowing the use of efficient 2-D CNNs or transformers for semantic segmentation. These methods are well-suited for real-time applications but can suffer from reduced spatial accuracy. Early methods like SqueezeSeg [14] and its improved version SqueezeSegV2 [15] focused on lightweight convolutional pipelines for range images. However, they struggled with blurry object boundaries and poor generalization, mainly due to their inability to preserve detailed 3-D spatial relationships. RangeNet++ [16] improved accuracy by using a DarkNet-based encoder and KNN-based postprocessing. Yet, it still relied on KNN interpolation, which added computational overhead and lacked theoretical robustness. Recent transformer-based methods like RangeViT [57] and RangeFormer [58] aim to address these issues by adding long-range attention in 2-D projections. However, they are still limited by the sparsity and discretization of the projected features, preventing the reconstruction of fine-grained geometry. Hybrid approaches like FRNet [56] and MPF [45] try to combine 3-D spatial context with 2-D projections. For example, MPF uses a multipath projection module to combine features from different viewpoints, which helps to reduce projection loss. However, these methods add computational complexity and require careful calibration across modalities.

In summary, point-based methods exhibit certain limitations in segmenting large-scale point clouds, resulting in suboptimal performance. On the other hand, voxel-based methods, while capable of providing higher accuracy, demand significant computational resources. In contrast, projection-based methods offer a more straightforward and faster approach. However, existing methods for processing projected images still have several shortcomings. For instance, overly simplistic design choices in the embedding process may lead to insufficient feature extraction. Furthermore, how to effectively utilize multilevel features in the backbone and enhance the segmentation of challenging objects remains a critical challenges that need addressing. This article focuses on these issues and proposes corresponding solutions.

III. Methodology

In this section, we first review the details of the range view in Section III-A. Then, we present the overall network architecture in Section III-B. To provide a better understanding of our method, we introduce the details of our network architecture, including the MSFE in Section III-C, the design of the backbone in Section III-D, the MFFM in Section III-E, the DSH in Section III-F, and the loss function in Section III-G.

A. Preliminaries

The LiDAR is mounted on the vehicle roof and performs a rotational scan to obtain a point cloud $P \in \mathbb{R}^{N\times 4}$. The point cloud consists of 3-D coordinates and reflection intensities. We project the point cloud P onto a 2-D image as follows:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \frac{1}{2}\left(1-\arctan\left(y,x\right)\pi^{-1}\right)W \\ 1-\left(\arcsin\left(z,d^{-1}\right)+l_d\right)l^{-1}H \end{pmatrix} \tag{1}$$

where $d=(x^2+y^2+z^2)^{1/2}$ is the depth of the point, $l=l_d+l_u$ denotes the vertical field of view (FOV) of the sensor, H and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

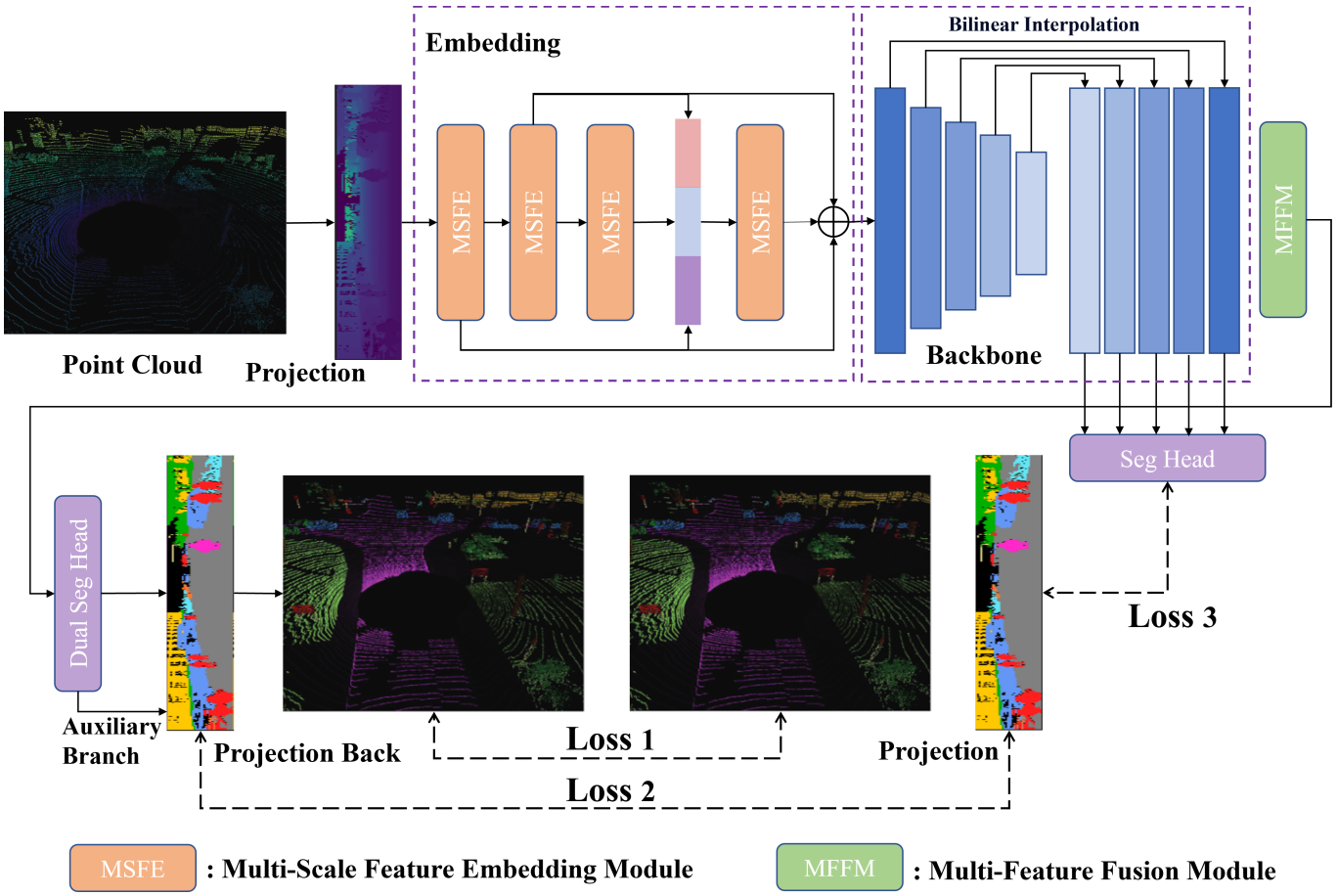


Fig. 4. Complete workflow of our proposed method. The process primarily consists of several components: the embedding layer, backbone, MFFM, and DSH. Initially, the method projects the point cloud into a RangeView image, followed by segmentation of the projected image, and ultimately reprojects the segmentation results back onto the original point cloud to complete the final segmentation. Notably, in the embedding layer, we designed an MSFE for extracting multiscale features from the projected image. After multilayer feature extraction in the backbone, MFFM effectively fuses these multilevel features, and the segmentation is completed using the DSH, enhancing the segmentation capability for challenging samples.

W are the height and width of the projected range image, and u and v represent the 2-D coordinates after the transformation. The image has five channels, which consist of the coordinate (x, y, z), depth of the point d, and the intensity information of the point r.

B. Network Architecture

As shown in Fig. 4, our network structure is presented. We will provide a detailed introduction to the proposed method, which includes the design of the MSFE, the construction of the backbone network, the implementation of the MFFM, and the design process of the DSH. In brief, our approach starts by projecting the input point clouds spherically to generate a range image, followed by feature extraction from this projected image. In the embedding layer, to maximize the feature extraction from the projected image, we propose the MFFM. This module effectively enhances feature extraction performance by employing multiscale feature extraction and multiscale feature channel mixing. Subsequently, in the backbone section, we process features through multilevel downsampling and feature reconstruction, utilizing the designed MFFM for the fusion of multilevel features, thereby achieving accurate segmentation. To improve the segmentation performance for difficult-to-classify objects, we have designed the DSH, which includes an auxiliary branch to assist the main segmentation head in achieving more accurate segmentation. We will discuss the design details of each module in depth.

C. MSFE Module

In general, before an image is fed into the backbone network, it first passes through an embedding layer to increase its dimensionality. Our experiments indicate that the design of the embedding layer has a significant impact on the final segmentation accuracy, as detailed in Section IV-F2. To capture richer multiscale features, we propose an MSFE. This module extracts multiscale features from the image using convolution kernels of different sizes and then combines these feature channels to fully integrate information from multiple scales. In the following, we will provide a detailed description of the processing flow of this module. As shown in Fig. 5, for the input image I, we choose three different size kernels (K_L, K_M, and K_S) to extract the features

$$\begin{pmatrix} F_{IL} \\ F_{IM} \\ F_{IS} \end{pmatrix} = \begin{pmatrix} \sigma(\mathrm{BN}(I \times K_L)) \\ \sigma(\mathrm{BN}(I \times K_M)) \\ \sigma(\mathrm{BN}(I \times K_S)) \end{pmatrix} \tag{2}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

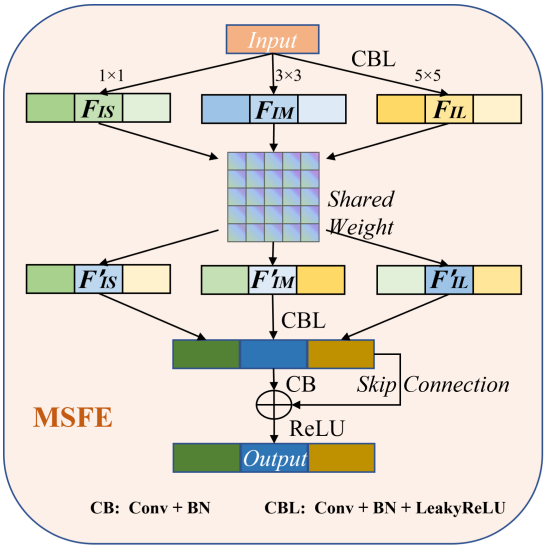


Fig. 5. MSFE module. This module extracts features using multiscale convolutions and combines multiscale features through channel splitting and recombination. In the output phase, skip connections are employed to prevent overfitting, effectively facilitating the extraction of multiscale information from the image.

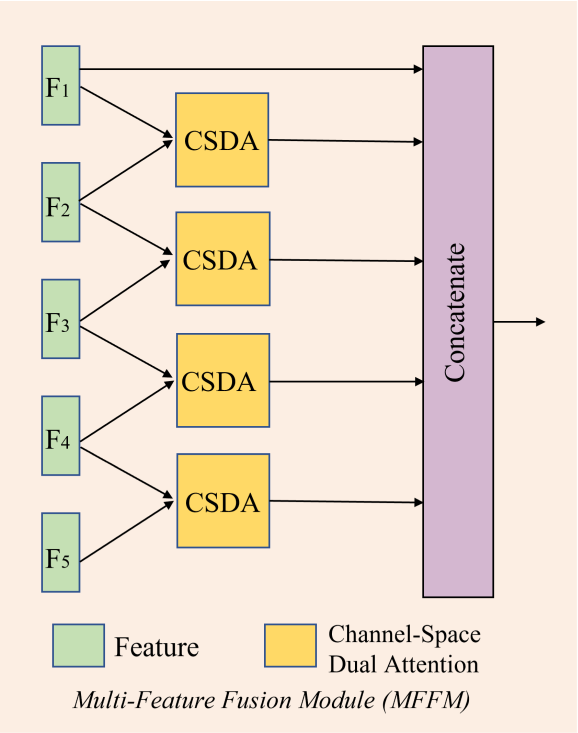


Fig. 6. MFFM. This module aims to thoroughly merge multilevel upsampled features. To achieve this, we have designed the CSDA module, which applies attention weighting to the fused features in both channel and spatial dimensions, effectively enhancing the feature fusion process.

where F_{IL}, F_{IM}, and F_{IS} represent the features extracted using the large kernel K_L, the middle kernel K_M, and the small kernel K_S; BN stands for batch normalization; σ denotes the ReLU activation function. Next, we mix the channels of these three features. Specifically, we divide the channels of each feature into three parts: front, middle, and back, then split and mix the channels of each feature. The mixed features are denoted as F'_{IL}, F'_{IM}, and F'_{IS}. Subsequently, we apply shared weights to further extract features from the mixed channels, which helps to reduce the number of parameters. The extracted features are then compressed by reducing the number of channels to one-third of the original features. This step successfully integrates the multiscale features, resulting in the mixed feature F_f, which is then subjected to a final feature fusion

$$F_{Mi} = \sigma\left(\mathrm{BN}\left(W \times F_f\right) + F_f\right) \tag{3}$$

where W represents the weights. The skip connection helps to prevent overfitting, and the final output feature of the MSFE module is denoted as F_{Mi}, where i indicates the ith MSFE module. In our embedding layer, we first use three MSFEs to extract features. Then, the output features from these three MSFE modules are concatenated and passed through another MSFE module for further feature fusion, resulting in the final feature F_{M4}. Therefore, the output feature of the embedding layer is

$$F_M = F_{M1} + F_{M2} + F_{M4} \tag{4}$$

where F_M is the output of the embedding layer.

D. Design of the Backbone

In the downsampling part of the backbone, we use the ResNet34 architecture to obtain the downsampled features F_{di}, where i denotes the number of downsampling operations. For these downsampled features, we perform image size reconstruction using bilinear interpolation. The upsampling process is described as follows:

$$F_{ui} = \sigma\left(\mathrm{BN}\left(\mathrm{BiInter}\left(W_i \times F_{di} + b_i\right)\right)\right) \tag{5}$$

where F_{ui} represents the upsampled reconstructed image features, BiInter$(\cdot)$ denotes the bilinear interpolation, and W_i and b_i are the learned weights and biases. In addition, BN represents batch normalization.

E. Multifeature Fusion Module

To effectively fuse multilevel features from the backbone, rather than relying solely on the last layer, we design the MFFM, as shown in Fig. 6. This module cross fuses multiple features to achieve comprehensive integration. In addition, we propose a channel-space dual attention (CSDA) method. We use channel separation and remixing, followed by calculating attention scores for the fused features in both channel and spatial dimensions. With the help of depthwise separable convolution, we further integrate the attention-enhanced features. Specifically, the processing flow of our MFFM is as follows:

$$F_{\mathrm{CS}i} = \mathrm{CSDA}\left(F_{ui}, F_{uj}\right), \quad j = i + 1, \ i = 1, 2, 3, 4. \tag{6}$$

Here, F_{ui} and F_{uj} represent the upsampled features obtained from the backbone, with F_{ui} and F_{uj} corresponding to different layers of the upsampled features. CSDA denotes the channel-space dual attention method that we proposed, which is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

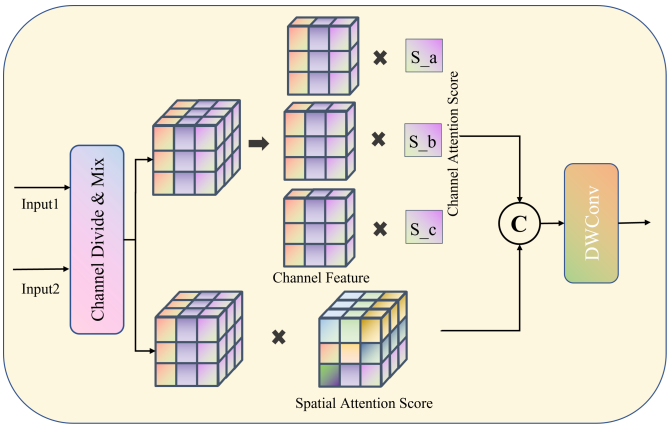


Fig. 7. CSDA. This module applies weighting to the original fused features in both channel and spatial dimensions. Subsequently, depthwise separable convolutions are utilized to further fuse the weighted features, achieving a comprehensive mixing of the input features.

introduced in the following paragraph, and F_{CSi} represents the output features

$$F_{\text{mffm}} = \text{concat}\left(F_M, F_{CS1}, F_{CS2}, F_{CS3}, F_{CS4}\right). \tag{7}$$

Here, F_{mffm} represents the fused multifeatures, and F_M denotes the input feature from the backbone layer. In addition, F_{CSi} represents the features processed by CSDA. After fusing the multilevel upsampled features, we fuse them with the output features from the embedding layer. This step not only prevents feature loss but also effectively enhances feature richness. Next, we will describe the processing flow of the CSDA module in detail.

1) Channel-Space Dual Attention: As shown in Fig. 7, we apply the same operations as in Section III-C to divide and mix the channel features of the two input features, forming the feature F_{dm}. For F_{dm}, we first perform a feature fusion using convolution and then calculate attention scores in both the channel and spatial dimensions. The operation in the channel dimension is as follows:

$$S = \text{Sigmoid}\left(f\left(\text{AAP}\left(F_{dm}\right)\right) + f\left(\text{AMP}\left(F_{dm}\right)\right)\right). \tag{8}$$

Here, AAP denotes the adaptive average pooling, AMP denotes the adaptive max pooling, and Sigmoid is the activation function. S represents the channel attention score. $f(\cdot)$ represents a composite operation consisting of multiple convolutional layers

$$F_C = S \times F_{dm}. \tag{9}$$

We multiply the feature F_{dm} by the channel attention score S, resulting in the channel feature F_C. The operation in the spatial dimension is as follows:

$$P_S = \text{Sigmoid}\left(f\left(\text{CAP}\left(F_{dm}\right)\right) + f\left(\text{CMP}\left(F_{dm}\right)\right)\right). \tag{10}$$

Here, CAP and CMP denote adaptive average pooling and adaptive max pooling in the spatial dimension, and P_S is the calculated spatial attention score. $f(\cdot)$ represents a composite operation consisting of multiple convolutional layers

$$F_S = P_S \times F_{dm}. \tag{11}$$

We multiply the spatial attention score P_S by the feature F_{dm} to obtain the spatial feature F_S. Finally, we use depthwise separable convolution to fuse the channel feature F_C and the spatial feature F_S

$$F_{\text{CS}} = \text{DWConv}\left(\text{Concat}\left(F_C, F_S\right)\right). \tag{12}$$

Here, F_{CS} is the final feature after applying dual attention weighting, and DWConv represents depthwise separable convolution.

F. Dual Segmentation Head

In our experiments, as shown in Fig. 2, we observed that the segmentation accuracy of certain categories, such as rider, truck, traffic sign, cone/stone, and pole in the SemanticPoss dataset [10], was significantly lower than that of other categories. As a result, we aim to improve the segmentation performance of these challenging categories. To address this issue, we designed a DSH to specifically enhance the segmentation of these difficult-to-segment objects. Specifically, we designed a separate segmentation head that treats these categories as positive samples and all other objects as negative samples. This head performs segmentation specifically for these categories, and the resulting output is used as an auxiliary branch to assist the main segmentation branch in improving its performance on these categories. The detailed steps are as follows:

$$F_{\text{seg}} = \alpha \cdot f_1\left(F_{\text{mffm}}\right) + \beta \cdot f_2\left(F_{\text{mffm}}\right) \tag{13}$$

where α and β are the training parameters. The auxiliary segmentation head is designed to segment hard objects, treating all other objects as having a 0 label.

G. Loss Function

In our system, an auxiliary segmentation approach is employed [19], where the segmentation loss is also applied to the upsampled features, denoted as $\text{loss}_{\text{aux}_1}$

$$\text{loss}_{\text{aux}_1} = \sum_{i=1}^{3} L\left(f\left(F_{ui}\right), \hat{y}\right). \tag{14}$$

Here, F_{ui} denotes the upsampled features, and $f(.)$ represents the multilayer perceptron (MLP) that outputs the segmentation from these features. $\hat{y}$ is the semantic label, and $L(.)$ refers to the cross-entropy loss. In addition, losses are computed both before and after the projection in the segmentation head, denoted as loss_1 and loss_2

$$\text{loss}_1 = L\left(F_{\text{seg}}, \hat{y}\right) + L_{\text{bd}}\left(F_{\text{seg}}, \hat{y}\right) + L_{\text{ls}}\left(F_{\text{seg}}, \hat{y}\right) \tag{15}$$

where F_{seg} is the output of segmentation head, and $L(.)$ refers to the cross-entropy loss. $L_{\text{ls}}(.)$ represents the Lovasz-Softmax loss [64], while $L_{\text{bd}}(.)$ corresponds to the boundary loss. In segmentation tasks, boundary blurring problems often arise due to upsampling and downsampling operations. To mitigate this issue, we employ a boundary loss function [60] defined as follows:

$$L_{\text{bd}}\left(\hat{y}, y\right) = 1 - \frac{2P^c R^c}{P^c + R^c} \tag{16}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

TABLE I

PERFORMANCE COMPARISON ON THE SEMANTICPOSS TEST. RED INDICATES THE BEST RESULT, PINK DENOTES THE SECOND BEST, AND BLUE REPRESENTS THE THIRD BEST

	person	rider	car	truck	plants	traffic sign	pole	trashcan	building	cone/stone	fence	bike	ground	mIoU
SqueezeSeg [14]	14.2	1.0	13.2	10.4	28.0	5.1	5.7	2.3	43.6	0.2	15.6	31.0	75.0	18.9
SqueezeSeg + CRF [14]	6.8	0.6	6.7	4.0	2.5	9.1	1.3	0.4	37.1	0.2	8.4	18.5	72.1	12.9
SqueezeSegV2 [15]	48.0	9.4	48.5	11.3	50.1	6.7	6.2	14.8	60.4	5.2	22.1	36.1	71.3	30.0
SqueezeSegV2 + CRF [15]	43.9	7.1	47.9	18.4	40.9	4.8	2.8	7.4	57.5	0.6	12.0	35.3	71.3	26.9
RangeNet53 [16]	55.7	4.5	34.4	13.7	57.5	3.7	6.6	23.3	64.9	6.1	22.2	28.3	72.9	30.3
RangeNet53 + KNN [16]	57.3	4.6	35.0	14.1	58.3	3.9	6.9	24.1	66.1	6.6	23.4	28.6	73.5	30.9
MINet [17]	61.8	12.0	63.3	22.2	68.1	16.3	29.3	28.5	74.6	25.9	31.7	44.5	76.4	42.7
MINet + KNN [17]	62.4	12.1	63.8	22.3	68.6	16.7	30.1	28.9	75.1	28.6	32.2	44.9	76.3	43.2
FIDNet-Point [18]	71.6	22.7	71.7	22.9	67.7	21.8	27.5	15.8	72.7	31.3	40.4	50.3	79.5	45.8
FIDNet-Point + KNN [18]	72.2	23.1	72.7	23.0	68.0	22.2	28.6	16.3	73.1	34.0	40.9	50.3	79.1	46.4
CENet [19]	74.9	21.8	77.0	25.3	72.0	18.0	30.9	46.9	75.9	26.1	47.5	51.7	80.7	49.9
CENet + KNN [19]	75.5	22.0	77.6	25.3	72.2	18.2	31.5	48.1	76.3	27.7	47.7	51.4	80.3	50.3
TransRVNet [20]	77.4	25.7	77.3	30.0	72.7	14.5	28.6	51.1	79.1	23.8	49.7	54.2	79.9	51.1
TransRVNet + KNN [20]	77.9	26.1	77.8	30.1	73.1	14.6	29.1	53.5	79.8	24.7	49.5	53.8	78.8	51.5
OPOC-MinkUNet14A [21]	63.8	33.6	75.7	26.3	69.3	9.6	32.2	-	79.2	-	36.6	54.0	71.5	50.2
Ours	75.9	24.6	78.8	29.2	71.8	26.0	33.7	53.4	77.7	40.4	49.5	52.4	80.8	53.4
Ours + KNN	75.8	25.1	79.4	29.3	72.3	26.0	34.6	55.9	78.4	41.1	48.0	51.3	78.8	53.6

where P^c and R^c represent the precision and recall of the predicted boundary map y_{pd} relative to the ground truth map y_{gt} for class c. The definition of boundaries is as follows:

$$\begin{cases} y^b_{\text{gt}} &= \text{pool}\left(1 - y_{\text{gt}}, \theta_0\right) - \left(1 - y_{\text{gt}}\right) \\ y^b_{\text{pd}} &= \text{pool}\left(1 - y_{\text{pd}}, \theta_0\right) - \left(1 - y_{\text{pd}}\right). \end{cases} \tag{17}$$

Here, pool(.) denotes the max-pooling operation, applied with a sliding window of size θ_0, which is set to 3. In addition, the loss after the projection in the segmentation head is defined as

$$\text{loss}_2 = L\left(\text{IProj}\left(F_{\text{seg}}\right), \hat{y_p}\right) \tag{18}$$

where IProj(.) denotes the back-projection operation and $\hat{y_p}$ represents the label of the ground truth point cloud. Furthermore, the loss for the auxiliary branch in the DSH is denoted as $\text{loss}_{\text{aux}_2}$, which is the loss function used by the auxiliary head to segment difficult samples, with the loss computed using cross-entropy loss. The total loss in our method is computed as

$$\text{Loss} = \text{Loss}_1 + \text{loss}_2 + a \cdot \text{loss}_{\text{aux}_1} + b \cdot \text{loss}_{\text{aux}_2}. \tag{19}$$

Here, a and b are the weights that balance the performance of the auxiliary branches. In our experiments, a and b are set to 1.

IV. EXPERIMENTS

A. Dataset

1) SemanticPOSS: The SemanticPOSS dataset [10] is collected by Peking University. It is a relatively compact but notably sparse benchmark, specifically designed for the evaluation of LiDAR-based scene understanding. It comprises 2988 distinct and intricate LiDAR scenes, each featuring numerous dynamic elements, such as pedestrians and cyclists, which add complexity to the task. This dataset is pointwise annotated and follows the same data structure as the SemanticKITTI dataset for consistency. The entire dataset is partitioned into six subsets, with Part 2 allocated as the test set, while the other five subsets serve as the training set.

2) SemanticKITTI: The SemanticKITTI dataset [11] is a large-scale benchmark designed for point cloud segmentation in autonomous driving scenarios. It consists of 43 551 LiDAR scans collected across 22 sequences from a German city, with each scan annotated with dense pointwise semantic labels. The dataset is split as follows: sequences 00–10, containing 19 130 scans, are used for training; sequence 08, with 4071 scans, serves as the validation set; and sequences 11–21, comprising 20 351 scans, are used for testing.

B. Evaluation Metric

For the segmentation evaluation in this method, we use the mean intersection over union (mIoU) for evaluation, calculated as follows:

$$\text{IoU}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i + \text{FN}_i} \tag{20}$$

$$\text{mIoU}_i = \frac{\sum_{i=1}^{M} \text{IoU}_i}{M} \tag{21}$$

where TP_i, FP_i, and FN_i represent the true positive, false positive, and false negative for the ith class, respectively. M represents the total number of classes.

C. Implementation Details

We conduct all the experiments on a single 4090 GPU. The stochastic gradient descent (SGD) optimizer with a momentum of 0.9 is used for network optimization. The loss functions include cross-entropy loss, Dice loss, Lovasz-Softmax loss [64], and boundary loss [60] to supervise the model training. For the SemanticPOSS dataset, we use standard augmentation techniques such as random rotation, random point cloud drop, and adding random point cloud noise. The network is trained for 60 epochs with a batch size of 6, and the projected image size is 64×512. The weight decay is set to $1e^{-4}$. For the SemanticKITTI dataset, we designed batch sizes of 2, 3, and 4, corresponding to projected image sizes of 64×512, 64×1024, and 64×2048, respectively. We train the network for 100 epochs with an initial learning rate of $1e^{-2}$ and a cosine annealing scheduler.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

TABLE II

SEMANTIC SEGMENTATION RESULTS ON SEMANTICKITTI TEST SET. THE BOLD ENTRIES INDICATE THE BEST RESULTS

Category	Methods	Input	mIoU	car	bicy	moto	truc	o.veh	ped	b.list	m.list	road	park	walk	o.gro	build	fenc	veg	trun	terr	pole	sign
Point-based	PointNet++ [23]	50K pts	20.1	53.7	1.9	0.2	0.9	0.2	0.9	1.0	0.0	72.0	18.7	41.8	5.6	62.3	16.9	46.5	13.8	30.0	6.0	8.9
	KPConv [28]		58.8	96.0	30.2	42.5	33.4	44.3	61.5	61.6	11.8	88.8	61.3	72.7	31.6	90.5	64.2	84.8	69.2	69.1	56.4	47.4
	PointASNL [59]		46.8	87.9	0.0	25.1	39.0	29.2	34.2	57.6	0.0	87.4	24.3	74.3	1.8	83.1	43.9	84.1	52.2	70.6	57.8	36.9
	BAAF [29]		**59.9**	95.4	31.8	35.5	48.7	46.7	49.5	55.7	53.0	90.9	62.2	74.4	23.6	89.8	60.8	82.7	63.4	67.9	53.7	52.0
	RandLa-Net [26]		53.9	94.2	26.0	25.8	40.1	38.9	49.2	48.2	7.2	90.7	60.3	73.7	20.4	86.9	56.3	81.4	61.3	66.8	49.2	47.7
	NeiEA-NET [54]		57.6	92.1	32.8	24.5	40.2	44.1	54.7	56.5	24.5	86.3	60.9	72.6	24.7	82.4	53.5	82.9	61.6	60.1	49.7	56.9
Voxel-based	MinkowskiNet-lite [31]	voxel	57.5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	MinkowskiNet [31]		63.1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	SPVCNN-lite [33]		58.5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	SPVCNN [33]		**63.8**	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Image-based	RangeNet++ [16]	64x512	41.9	87.4	26.2	26.5	18.6	15.6	31.8	33.6	4.0	**91.4**	57.0	74.0	26.4	81.9	52.3	77.6	48.4	63.6	36.0	50.0
	MPF [45]		48.9	91.1	22.0	19.7	18.8	16.5	30.0	36.2	4.2	91.1	61.9	74.1	29.4	86.7	56.2	**82.3**	51.6	68.9	38.6	49.8
	FIDNet-Point [18]		51.3	90.4	28.6	30.9	34.3	27.0	43.9	48.9	16.8	90.1	58.7	71.4	19.9	84.2	51.2	78.2	51.9	64.5	32.7	50.3
	CENet [19]		60.7	92.1	45.4	42.9	43.9	**46.8**	56.4	63.8	29.7	91.3	66.0	**75.3**	31.1	88.9	**60.4**	81.9	60.5	67.6	49.5	59.1
	Ours		**63.3**	**92.7**	**48.8**	**46.9**	**50.1**	45.3	**62.7**	**68.7**	**35.4**	91.3	**66.6**	74.3	**36.2**	**89.4**	59.8	82.1	**66.1**	**69.5**	**57.2**	**60.0**
	RangeNet++ [16]	64x1024	48.0	90.3	20.6	27.1	25.2	17.6	29.6	34.2	7.1	90.4	52.3	72.7	22.8	83.9	53.3	77.7	52.5	63.7	43.8	47.2
	MPF [45]		53.6	92.7	28.2	30.5	26.9	25.2	42.5	45.5	9.5	90.5	64.7	74.3	32.0	88.3	59.0	83.4	56.6	69.8	46.0	54.9
	FIDNet-Point [18]		56.0	92.4	44.0	41.5	33.2	30.8	57.9	52.6	18.0	91.0	61.2	73.8	12.6	88.2	57.9	80.8	59.5	65.1	45.3	58.4
	CENet [19]		62.3	93.0	**50.5**	47.6	41.7	43.4	64.5	65.2	32.5	90.5	**65.5**	74.1	29.2	**90.9**	65.4	81.6	65.4	65.6	55.9	**61.0**
	Ours		**65.2**	**93.3**	48.5	**51.9**	**53.3**	**55.3**	**67.9**	**73.5**	**37.5**	**91.8**	63.6	**76.9**	**37.0**	90.6	**65.5**	**85.4**	**67.1**	**69.8**	**62.8**	46.4
	SqueezeSeg-CRF [14]	64x2048	30.8	68.3	18.1	5.1	4.1	4.8	16.5	17.3	1.2	84.9	28.4	54.7	4.6	61.5	29.2	59.6	25.5	54.7	11.2	36.3
	SqueezeSegV2-CRF [15]		39.6	82.7	21.0	22.6	14.5	15.9	20.2	24.3	2.9	88.5	42.4	65.5	18.7	73.8	41.0	68.5	36.9	58.9	12.9	41.0
	SqueezeSegV3 [38]		55.9	92.5	38.7	36.5	29.6	33.0	45.6	46.2	20.1	91.7	63.4	74.8	26.4	89.0	59.4	82.0	58.7	65.4	49.6	58.9
	SalsaNext [46]		59.5	91.9	48.3	38.6	38.9	31.9	60.2	59.0	19.4	91.7	63.7	75.8	29.1	90.2	64.2	81.8	63.6	66.5	54.3	62.1
	KPRNet [44]		63.1	**95.5**	54.1	47.9	23.6	42.6	65.9	65.0	16.5	**93.2**	**73.9**	80.6	30.2	91.7	68.4	85.7	69.8	71.2	58.7	64.1
	RangeNet++ [16]		52.2	91.4	25.7	34.4	25.7	23.0	38.3	38.8	4.8	91.8	65.0	75.2	27.8	87.4	58.6	80.5	55.1	64.6	47.9	55.9
	MPF [45]		55.5	93.4	30.2	38.3	26.1	28.5	48.1	46.1	18.1	90.6	62.3	74.5	30.6	88.5	59.7	83.5	59.7	69.2	49.7	58.1
	FIDNet-Point [18]		58.6	93.0	45.7	42.0	27.9	32.6	62.6	58.1	30.5	90.8	58.3	74.9	20.1	88.5	59.5	83.1	64.3	67.8	52.6	60.0
	CENet [19]		64.7	91.9	58.6	50.3	40.6	42.3	68.9	65.9	43.5	90.3	60.9	75.1	31.5	91.0	66.2	84.5	69.7	70.0	61.5	67.6
	LiteHDSeg [60]		63.8	92.3	40.0	55.4	37.7	39.6	59.2	71.6	**54.3**	93.0	68.2	78.3	29.3	91.5	65.0	78.2	65.8	65.1	59.5	67.7
	RangeViT [57]		64.0	95.4	55.8	43.5	29.8	42.1	63.9	58.2	38.1	93.1	70.2	80.0	32.5	**92.0**	**69.0**	85.3	70.6	71.2	60.8	64.7
	LENet [62]		64.5	93.9	57.0	51.3	44.3	44.4	66.6	64.9	36.0	91.8	68.3	76.9	30.5	91.2	66.0	83.7	68.3	67.8	58.6	63.2
	MSCF [63]		55.2	90.7	46.1	34.6	30.5	19.8	55.6	52.0	10.8	90.5	55.0	73.5	26.6	86.6	56.3	82.0	60.4	66.4	51.1	59.9
	TFNet [61]		66.1	94.3	**60.7**	**58.5**	38.4	48.4	74.3	72.2	35.5	90.6	68.5	75.3	29.0	91.6	67.3	83.8	**71.1**	67.0	60.8	**68.7**
	Ours		**67.8**	93.6	57.5	58.4	**50.9**	**53.6**	**74.3**	**74.6**	41.5	91.7	60.0	**82.0**	**35.1**	91.0	64.0	**86.2**	70.6	**72.2**	**63.3**	67.2

D. Comparisons With State-of-the-Art Methods

1) SemanticPOSS: As shown in Table I, we present the segmentation performance of our method on the SemanticPOSS dataset. Compared with several state-of-the-art approaches, our method achieves significant improvements across various categories. Notably, our method consistently outperforms existing methods in the segmentation of challenging objects, including rider, truck, traffic sign, cone/stone, and pole. This demonstrates our method's ability to handle difficult segmentation tasks that other approaches struggle with. Our method achieves an overall mIoU of 53.4%, surpassing the best-performing method, TransRVNet [20], by 1.5%. On the SemanticKITTI dataset, our method achieves an mIoU of 53.6%, which is 1.8% higher than TransRVNet + KNN [20], further validating the robustness and effectiveness of our approach. For difficult-to-segment objects, such as rider and truck, which are small in size and share similar appearances with other objects, our method achieves 24.6% and 29.2%, respectively. These results are among the highest, significantly outperforming other methods like SqueezeSeg [14] and RangeNet53 [16], which struggle with these categories. For the traffic sign category, which is often confused with background elements, our method achieves 26.0% accuracy, surpassing the previous best method, and TransRVNet with an accuracy of 25.7%. This demonstrates that our method excels in handling small and cluttered objects. In addition, for cone/stone and pole, which are frequently occluded or only partially visible, our method shows clear advantages, achieving 40.4% and 49.5% accuracy, respectively. These results are significantly better than those of other methods, including MINet [17] and CENet [19], which perform poorly on these challenging categories.

2) SemanticKITTI: In Table II, wepresent a performance comparison of various methods on the dataset. Our proposed method achieves an overall mIoU of 67.8%, demonstrating superior performance among the different approaches, particularly in various categories of segmentation. Specifically, compared with the benchmark method KPRNet (mIoU of 63.1%) [44], our method improves by 4.7 percentage points, illustrating the strong capability of our model in handling complex scenes. Notably, we achieved segmentation accuracies of 50.9%, 53.6%, and 74.3% for the categories "truck," "other-vehicle," and "person," respectively, significantly surpassing other existing methods. Furthermore, our approach attained a high score of 86.2% in the "vegetation" category, outperforming KPRNet and other traditional image-based methods. In addition, compared with other image-based segmentation algorithms such as RangeNet++ [16] and MPF [45], which achieved mIoUs of 52.2% and 55.5% at an input size of 64×2048, our method demonstrates a remarkable improvement, underscoring its overall effectiveness. Particularly in

TABLE III

COMPLEXITY ANALYSIS OF DIFFERENT METHODS ON SEMANTICPOSS

Methods	Param.(M)	Mean Inference Time (s)	mIoU (%)
CENet [19]	6.8	0.03	49.9
FIDNet [18]	6.0	0.04	45.8
SqueezeSegV3 [38]	9.2	0.02	41.9
Ours	6.8	0.05	53.4

TABLE IV

MODULE ABLATION (MSFE, MFFM, AND DSH)

MSFE	MFFM	Dual Seg Head	mIoU (POSS)	mIoU (KITTI)
×	×	×	50.5	60.9
✓	×	×	51.7	61.8
✓	✓	×	52.6	62.7
✓	✓	✓	**53.6**	**63.3**

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

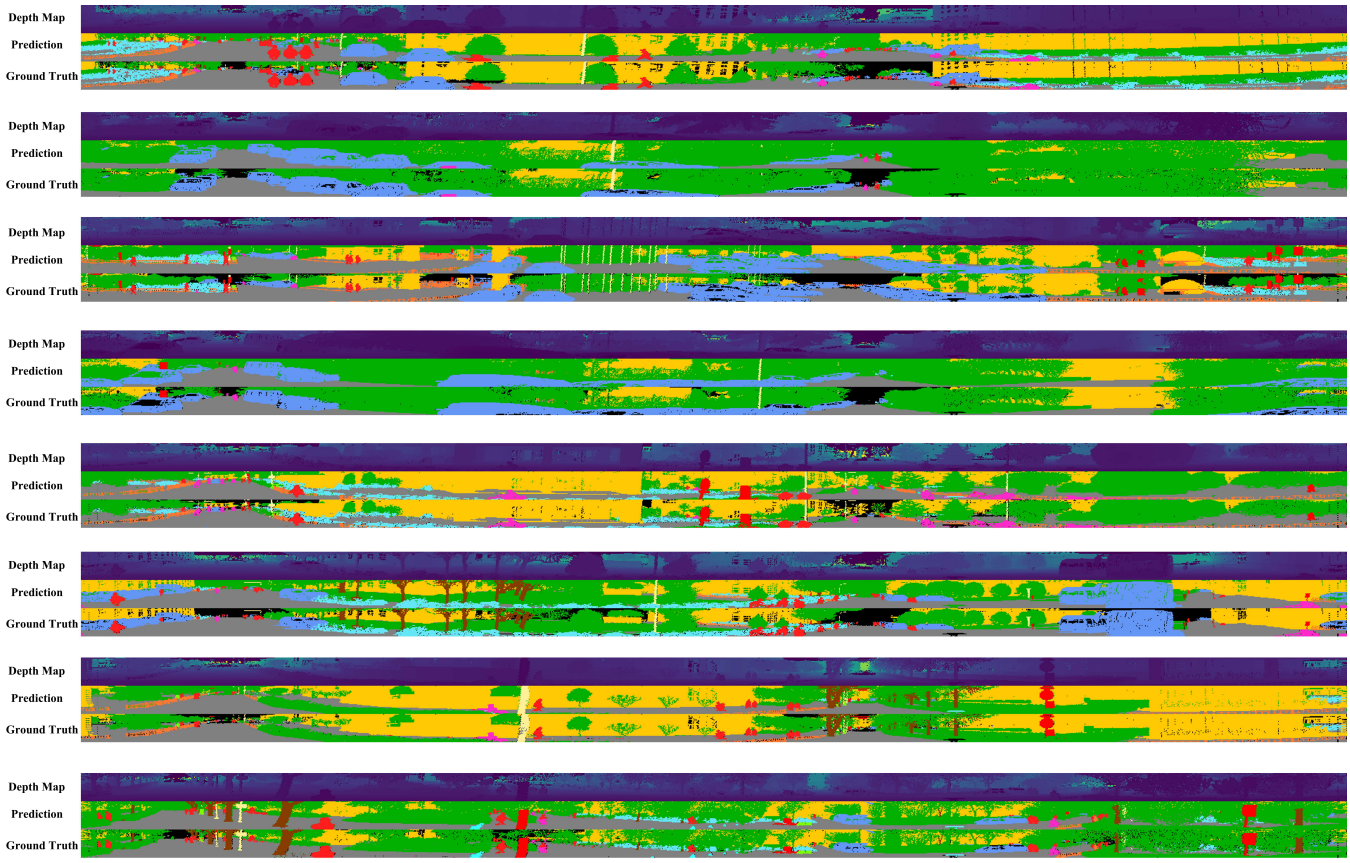


Fig. 8. Visualization of segmentation results on the projected images. We compared the segmentation results of the projected images with the ground truth on the SemanticPOSS dataset. The results indicate that our method performs well in segmenting the ground and pedestrians, with segmentation contours being quite accurate. In addition, the segmentation results for small objects are also close to the ground truth, demonstrating the effectiveness of our method in capturing fine details.

challenging scenarios, our method exhibits superior detection rates for small objects, reflecting higher segmentation accuracy and granularity. The results indicate that our method excels in multiclass segmentation tasks, with all performance metrics showing significant enhancement, showcasing its potential for practical applications.

3) Visualization: We present three visualizations of the segmentation results. Fig. 8 shows the visualization of the RangeView segmentation results. Fig. 9 presents the point cloud segmentation results, which are compared with the ground truth, SqueezeSeg [14], SqueezeSegV2 [15], SqueezeSegV3 [38], CENet [19], and MINet [17]. Fig. 10 provides a zoomed-in view of the segmentation results from Fig. 9, allowing for a more detailed analysis and comparison.

In Fig. 8, we compared the projection-based segmentation results with the ground truth. The results show that the segmentation of "person," "vehicle," "road," and "building" is nearly identical to the ground truth, with an overall excellent segmentation performance. Particularly, the segmentation of person and vehicle contours is notably accurate, and even the segmentation of distant persons is well-preserved, which demonstrates the high accuracy of our method. Fig. 9 presents the reprojected segmentation results onto the 3-D point cloud, which are compared with the ground truth, SqueezeSeg, SqueezeSegV2, SqueezeSegV3, CENet, and MINet. Overall, our method produces the closest results to the ground truth. In comparison, SqueezeSeg and SqueezeSegV2 exhibit significantly poorer performance in ground segmentation. In addition, SqueezeSegV3, CENet, and MINet underperform in segmenting trees and small objects, such as people, compared with our method. This not only highlights the effectiveness of our approach but also demonstrates its strong capability in segmenting both large and small objects. Fig. 10 shows a zoomed-in view of the segmentation results for a more detailed observation. The areas with outstanding segmentation performance are highlighted with red boxes. From the results, it is clear that our method has a significant advantage in segmenting the ground, pedestrians, and trees. Moreover, our method demonstrates a strong ability to segment distant ground and trees, further validating the superior performance of our approach. In Fig. 11, we present a comparison of the segmentation results for several challenging samples. Our method is compared with SqueezeSeg, SqueezeSegV2, SqueezeSegV3, CENet, and MINet. The regions where our method excels are highlighted with rectangular boxes, while the areas where other methods underperform are indicated with circular boxes. It is evident that our approach delivers superior results on these challenging samples, particularly in segmenting the contours of vehicles, pedestrians, bicycles, vegetation, and distant vehicle outlines. These results further underscore the effectiveness of our approach.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

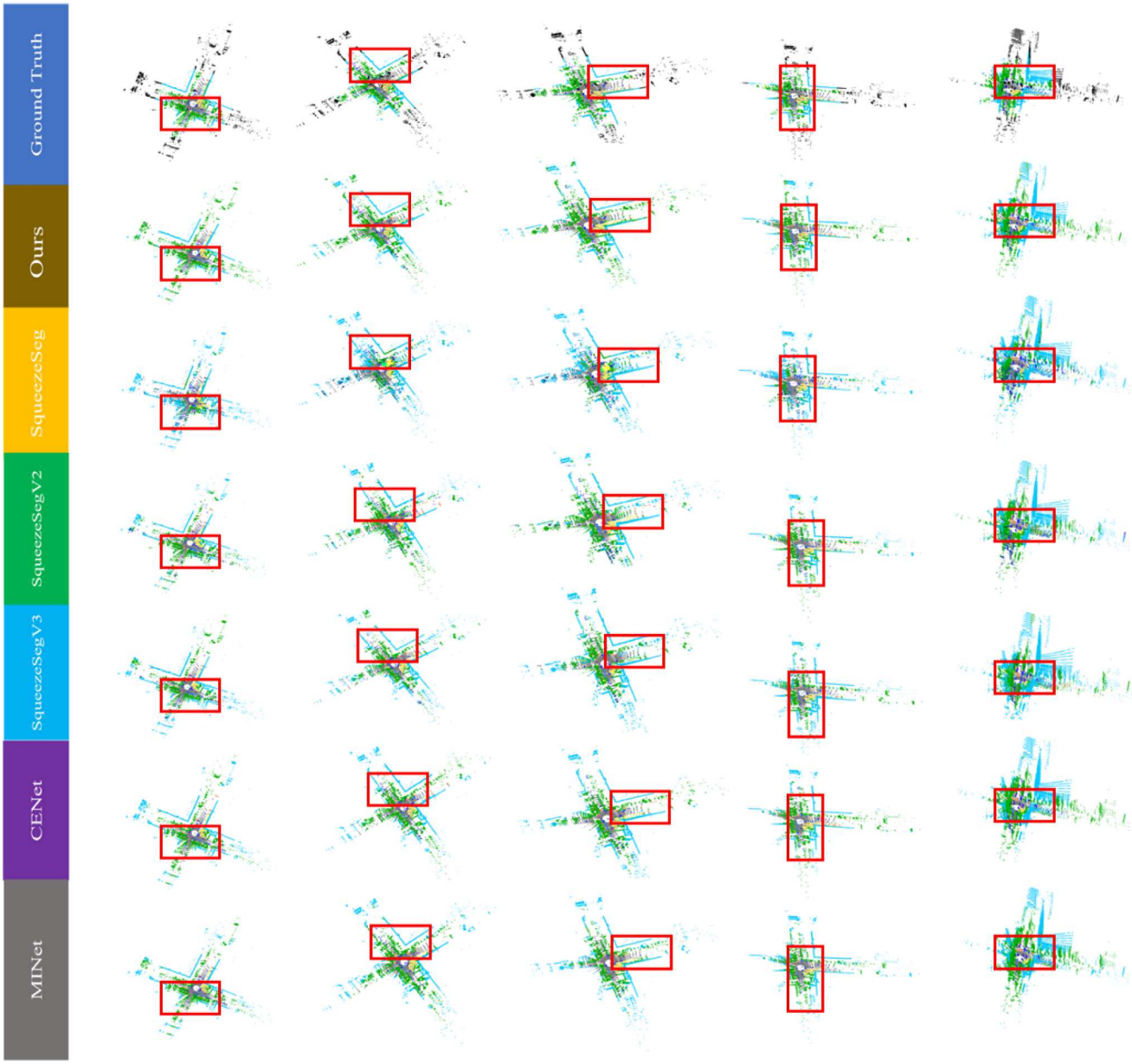


Fig. 9. We visualized the segmentation results on the SemanticKITTI dataset and compared them with the ground truth as well as the results from SqueezeSeg [14], SqueezeSegV2 [15], SqueezeSegV3 [38], CENet [19], and MINet [17]. It is evident that our method achieves a segmentation performance that is highly consistent with the ground truth for the ground category. Furthermore, for the segmentation of roadside trees and pedestrians, our method also demonstrates the closest alignment with the ground truth, highlighting its outstanding performance. The areas marked in red indicate where our method significantly outperforms others.

E. Computational Cost and Inference Speed

As shown in Table III, a systematic comparison was conducted between our proposed method and several benchmark approaches. Inference was performed on the SemanticPOSS validation set to compute the average inference speed and compare the model parameters. Experimental results indicate that, while significantly enhancing accuracy, our method incurs only a modest increase in the number of parameters and maintains an inference speed that satisfies real-time requirements.

F. Ablation Study

1) Module Ablation Study: We conducted ablation experiments on the proposed MSFE, MFFM, and DSH. The experimental results are shown in Table IV, validated on the SemanticPOSS and SemanticKITTI datasets. According to the results, the integration of all modules significantly improved the model's performance. Specifically, when no modules are utilized, the mIoU is 50.5% on the SemanticPOSS dataset and 60.9% on the SemanticKITTI dataset. The introduction of MSFE yields a slight performance improvement, with mIoU increased to 51.7% and 61.8%, indicating that MSFE can initially enhance feature extraction. Furthermore, with the addition of MFFM, the mIoU rises significantly to 52.6% (on POSS) and 62.7% (on KITTI), demonstrating the critical role of multilevel feature fusion in improving segmentation performance. Finally, when all modules are employed,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

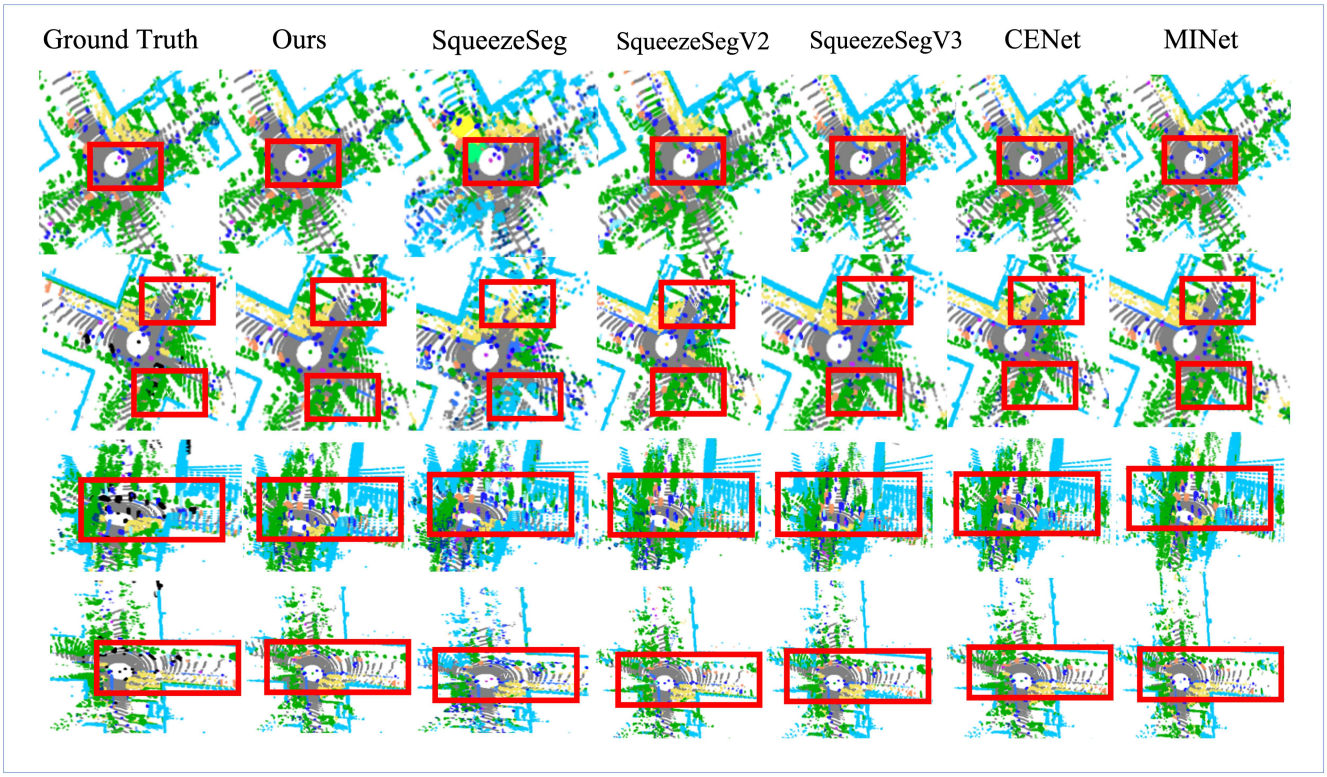


Fig. 10. Detailed performance of the segmentation results in an enlarged view. The areas marked in red indicate where our method significantly outperforms others. It is evident that our approach demonstrates excellent performance in both ground and pedestrian segmentation.

TABLE V
PERFORMANCE OF THE MSFE ACROSS DIFFERENT METHODS

Methods	mIoU (%)
CENet	49.9
CENet (MSFE)	**51.5 (+1.6)**
RangeNet++	30.3
RangeNet++ (MSFE)	**32.4 (+2.1)**
MINet	42.7
MINet (MSFE)	**43.9 (+1.2)**
SqueezeSeg	18.9
SqueezeSeg (MSFE)	**34.6 (+15.7)**
SqueezeSegV2	30.0
SqueezeSegV2 (MSFE)	**41.9 (+11.9)**

the mIoU reaches 53.6% (POSS) and 63.3% (KITTI), showcasing the overall superiority of our method. These results validate the effectiveness of each proposed module in point cloud segmentation and highlight their complementary roles in enhancing segmentation accuracy.

2) Effectiveness of the MSFE: In Table V, we present the impact of the MSFE on the performance of various models. The results indicate that the incorporation of MSFE significantly enhances the performance of existing models. Specifically, the baseline mIoU for CENet is 49.9%, which is improved to 51.5% with the addition of MSFE, resulting in a gain of 1.6 percentage points. This improvement suggests that MSFE effectively enhances the model's performance during feature extraction. Similarly, RangeNet++ shows an original mIoU of 30.3%, which rises significantly to 32.4% upon the integration of MSFE, achieving an increase of 2.1 percentage points. This further confirms the effectiveness of MSFE in augmenting the performance of lower performing models. For MINet, the original mIoU is 42.7%, and with the application of MSFE, it increases to 43.9%, yielding a gain of 1.2 percentage points. Although this increase is relatively modest, it still reflects the value of MSFE in improving feature extraction. For SqueezeSeg, the baseline mIoU is 18.9%, which increases significantly to 34.6% with the integration of MSFE, resulting in a gain of 15.7 percentage points. Also, SqueezeSegV2 shows an initial mIoU of 30.0%, which rises substantially to 41.9% following the application of MSFE, yielding a gain of 11.9 percentage points. These results underscore the substantial impact of MSFE in enhancing model performance across a broad spectrum of baseline performances. Overall, the MSFE module enhances the multidimensional representation of features, improving the model's performance in point cloud segmentation tasks. These results validate the effectiveness of the MSFE module we designed, further highlighting the importance of the embedding layer's design and providing critical insights for future model improvements.

3) Effectiveness of the MFFM: In Table VI, we present the impact of the MFFM on the performance of various models. The results indicate that the incorporation of MFFM effectively enhances the performance of existing models. Specifically, the baseline mIoU for CENet is 49.9%, which rises to 50.8% with the addition of MFFM, resulting in a gain of 0.9 percentage points. This suggests that MFFM significantly improves the model's performance through effective feature fusion. For SqueezeSeg, the original mIoU is 18.9%,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

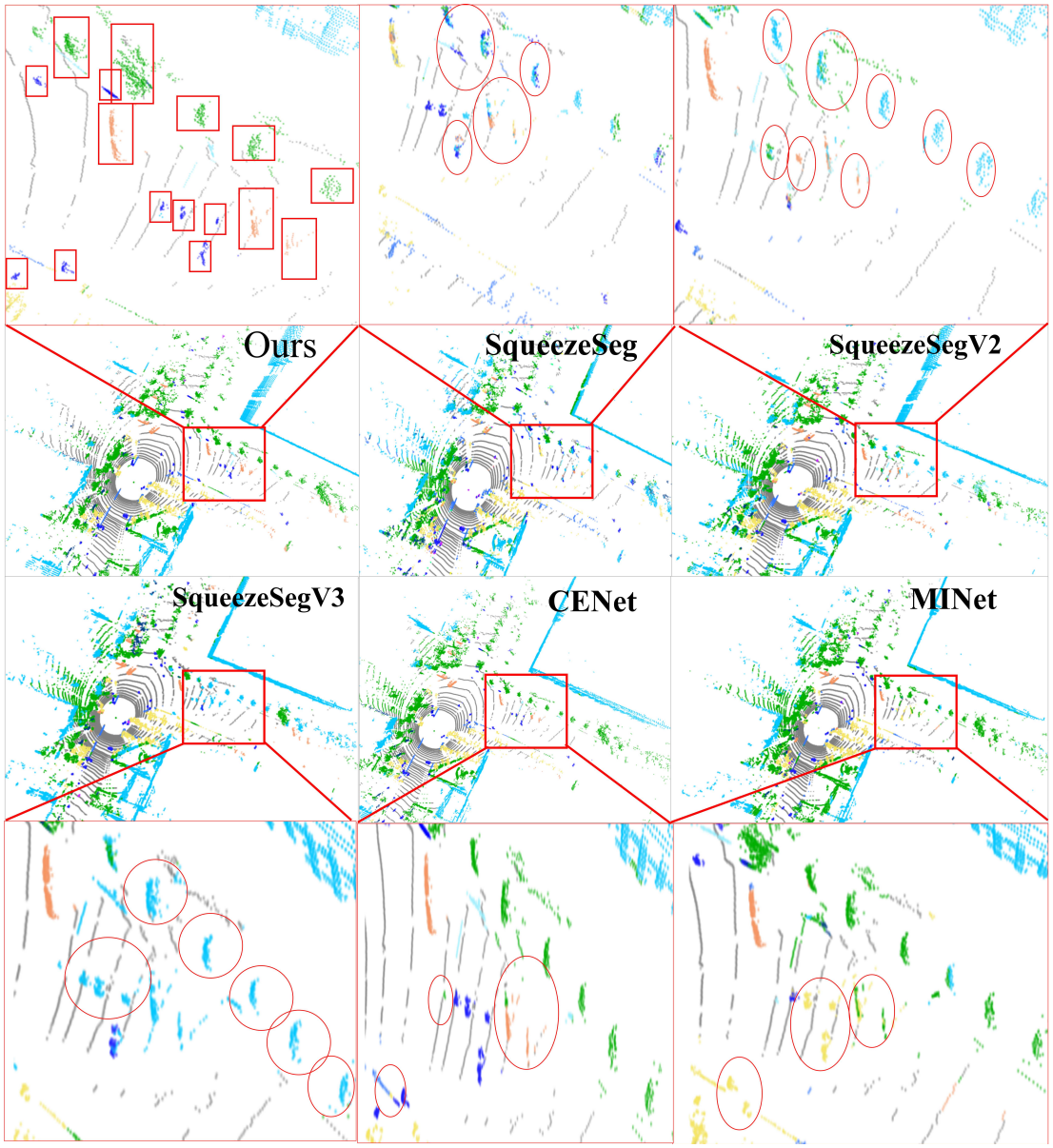


Fig. 11. We compare the segmentation results of some challenging samples across different methods. The samples segmented by our method are highlighted with rectangular boxes, while the areas of segmentation failure in other methods are marked with circular boxes.

TABLE VI

COMPARISON OF MIOU BEFORE AND AFTER INTEGRATING THE MFFM. THIS TABLE PRESENTS THE CHANGES IN MIOU FOR THE CENET, SQUEEZESEG, MINET, RANGENET++, AND SQUEEZESEGV2 MODELS UPON THE ADOPTION OF MFFM, HIGHLIGHTING ITS IMPACT ON MODEL PERFORMANCE

Methods	mIoU (%)
CENet	49.9
CENet (MFFM)	**50.8 (+0.9)**
RangeNet++	30.3
RangeNet++ (MFFM)	**32.1 (+1.8)**
MINet	42.7
MINet (MFFM)	**44.7 (+2.0)**
SqueezeSeg	18.9
SqueezeSeg (MFFM)	**20.4 (+1.5)**
SqueezeSegV2	30.0
SqueezeSegV2 (MFFM)	**31.2 (+1.2)**

and with the integration of MFFM, it increases to 20.4%, achieving a gain of 1.5 percentage points. This underscores the effectiveness of MFFM in enhancing lower performing models. Similarly, SqueezeSegV2 shows an original mIoU of 30.0%, which improves to 31.2% upon the application of MFFM, resulting in a gain of 1.2 percentage points. Although this increase is relatively modest, it nonetheless highlights the important role of MFFM in feature fusion. In addition, for RangeNet++, the baseline mIoU is 30.3%, which increases to 32.1% with MFFM, yielding a gain of 1.8 percentage points. For MINet, the baseline mIoU is 42.7%, and with MFFM, it rises to 44.7%, achieving a gain of 2 percentage points. These results further underscore the effectiveness of MFFM in enhancing model performance across various performance tiers. Overall, the MFFM module enhances model performance in point cloud segmentation tasks by effectively merging features from different levels. These results validate the effectiveness of the MFFM module we designed.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
IMPACT OF DIFFERENT CONVOLUTION TYPES ON CSDA

	m.IoU (SemanticPOSS)	m.IoU (SemanticKITTI)
Regular Convolution	51.7	62.5
Depthwise Separable Convolution	**53.4(+1.7)**	**63.8(+1.3)**

TABLE VIII
COMPARISON OF SEGMENTATION ACCURACY FOR SINGLE SEGMENTATION HEAD AND DSH ACROSS DIFFERENT CATEGORIES

Structure	rider	truck	traffic sign	pole	cone/stone
With Single Seg Head	23.8	27.2	25.5	32.7	38.9
With Dual Seg Head	**25.1 (+1.3)**	**29.3 (+2.1)**	**26.0 (+0.5)**	**34.6 (+1.9)**	**41.1 (+2.2)**

TABLE IX
GENERALIZATION TEST OF THE DSH. WE EVALUATE THE GENERALIZATION CAPABILITY OF THE PROPOSED DSH ON THE SHAPENETPART PART SEGMENTATION DATASET BY APPLYING IT TO OTHER BASELINE METHODS

Methods	class. mIoU	aerphone	motorbike	rocket
PointMLP [47]	82.2	75.8	71.7	64.7
PointMLP (DSH)	**83.8 (+1.6)**	**78.6 (+2.8)**	**74.3 (+2.6)**	**65.5 (+0.8)**
DGCNN [48]	82.3	74.7	66.3	63.5
DGCNN (DSH)	**83.7 (+1.5)**	**76.1 (+1.4)**	**68.2 (+1.9)**	**64.7 (+1.2)**
PointNet++ [23]	81.7	75.8	68.2	59.1
PointNet++ (DSH)	**82.8 (+1.1)**	**76.8 (+1.0)**	**69.8 (+1.6)**	**60.5 (+1.4)**

4) Effect of Depthwise Separable Convolution in CSDA: In the CSDA module, we employed depthwise separable convolutions. Compared with conventional convolutions, depthwise separable convolutions exhibited superior performance in our experiments. Detailed experimental results are provided in Table VII. We evaluated the model on both the SemanticPOSS and SemanticKITTI datasets. The results demonstrate that, relative to conventional convolutions, the use of depthwise separable convolutions resulted in a 1.7% improvement on SemanticPOSS and a 1.3% improvement on SemanticKITTI. These findings validate the effectiveness of depthwise separable convolutions within the CSDA module.

5) Effectiveness of the DSH: In Table VIII, we present the performance impact of the DSH compared with the single segmentation head across several challenging categories. The results indicate that the introduction of the DSH significantly improves the model's segmentation accuracy in these difficult categories. Specifically, with the single segmentation head, the segmentation accuracy for the "rider" category is 23.8%, which increases to 25.1% with the DSH, resulting in a gain of 1.3 percentage points. For the "truck" category, the original accuracy is 27.2%, which improves to 29.3% when using the DSH, achieving an increase of 2.1 percentage points. This demonstrates the DSH's substantial enhancement of segmentation capabilities for these challenging objects. In the "traffic sign" category, the accuracy with the single segmentation head is 25.5%, which rises to 26.0% with the DSH, yielding a gain of 0.5 percentage points. Although this increase is relatively modest, it still reflects the improvement brought about by the DSH. For the "pole" and "cone/stone" categories, accuracies rise from 32.7% to 34.6% (an increase of 1.9%) and from 38.9% to 41.1% (an increase of 2.2%), respectively, demonstrating the effectiveness of the DSH across various object categories.

TABLE X
IMPACT OF a AND b IN LOSS FUNCTION

a	0	0.5	1.0
mIoU (%)	47.7	52.5	52.8
b	0	0.5	1.0
mIoU (%)	52.8	53.1	53.4

Furthermore, we assess the generalization capability of the DSH across various datasets and models to evaluate its effectiveness in mitigating segmentation discrepancies caused by class imbalance. Specifically, we incorporate DSH into PointMLP [47], DGCNN [48], and PointNet++ [23] on the ShapeNetPart dataset [49]. Auxiliary segmentation is performed on three imbalanced categories: airplane, motorbike, and rocket. As illustrated in Table IX, DSH consistently enhances segmentation performance across these categories, highlighting its generalizability and its effectiveness in alleviating the impact of class imbalance in diverse settings.

Overall, the DSH enhances the model's performance on challenging categories by utilizing auxiliary branches for segmentation. These results further validate the effectiveness of the DSH, allowing for more balanced segmentation results, particularly enhancing performance on difficult categories.

6) Effects of a and b in Loss Function: In the context of the loss function, experiments were conducted to determine the optimal values of a and b. The results are shown in Table X. Initially, we tested various values of a, specifically 0, 0.5, and 1, to validate their effectiveness. It was determined that $a = 1$ yielded the optimal results. Building upon this, additional experiments were conducted to identify the optimal value of b, with values of 0, 0.5, and 1 tested. Ultimately, the experiments demonstrated that $b = 1$ yielded the best performance. Consequently, in our proposed method, both a and b are set to 1.

V. DISCUSSION AND FUTURE WORKS

A. Choice of Projection View

In this study, we employ a spherical projection that aligns with the LiDAR emission direction, effectively preserving object contours and structural features (see Fig. 8). This method maintains spatial integrity and minimizes geometric distortion, providing a comprehensive view of objects, especially in complex environments with prevalent occlusions. Unlike orthogonal projections that can obscure vertical structures, spherical projections retain relative depth information, facilitating the accurate delineation of objects like pedestrians and vegetation, which is crucial in urban scenarios. However, while spherical projection minimizes distortion when the sensor is stationary, its effectiveness may decrease in dynamic environments with varied scanning patterns. Hybrid projection strategies that integrate both spherical and planar methods could help to adapt these conditions, enhancing scene understanding. Comparative analysis indicates that although spherical projections excel in preserving structural details, they may struggle with objects near the edges of the spherical FOV, leading to potential feature loss. Addressing these tradeoffs requires further investigation into optimal projection angles

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

and dynamic adjustment mechanisms. In conclusion, while the spherical projection method offers significant advantages for preserving object features in LiDAR data, its implementation must consider the specific requirements of the application domain, focusing on algorithmic improvements and complementary techniques for enhanced performance across various operational environments.

B. Application Scenarios

The proposed method is highly effective for point cloud segmentation tasks, demonstrating significant efficiency that allows for deployment on various edge devices, including drones and autonomous robots (see Table III). This versatility enables applications in ground robot navigation, autonomous vehicle path planning, and trajectory optimization for aerial drones, enhancing environmental awareness and decision-making. However, in scenarios where high segmentation accuracy is critical—such as medical imaging or dense urban environments—voxel-based methods may provide superior performance. These methods excel in capturing intricate details when computational resources are ample. Future research should focus on improving the precision of our approach to meet the demands of high-stakes applications. Exploring hybrid techniques that integrate the strengths of both point cloud and voxel-based methods can enhance overall segmentation accuracy while leveraging our method's efficiency. By addressing these areas, we can ensure that our approach remains relevant and effective across a broader range of challenging applications.

C. Application of Physical Principles

While the proposed method primarily leverages deep learning, we enhance its robustness through data augmentation by introducing a small amount of Gaussian noise into the point cloud data. This approach effectively simulates the inherent noise present in real-world scenarios, improving the model's generalization capabilities. Looking ahead, our research will focus on minimizing information loss during the projection and back-projection processes, which are critical steps in the point cloud processing pipeline. We also aim to integrate additional physical models to further enhance segmentation performance. By incorporating principles from physics with deep learning techniques, we aspire to achieve more accurate and reliable segmentation results, particularly in complex and dynamic environments where traditional methods may struggle.

VI. Conclusion

This article presents an enhanced projection-based approach for LiDAR point cloud semantic segmentation, introducing the MSFE and MFFM to improve feature extraction and fusion. In addition, a DSH is proposed to enhance the segmentation accuracy for difficult-to-detect objects. Experimental results on the SemanticPOSS and SemanticKITTI datasets demonstrate that the proposed method outperforms state-of-the-art techniques, particularly in segmenting small and occluded objects, highlighting its practical applicability.

References

[1] Z. He, J. Liu, and S. Yang, "DE-Net: A dual-encoder network for local and long-distance context information extraction in semantic segmentation of large-scale scene point clouds," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 15914–15926, 2024.

[2] Y. Duan et al., "MFSA-Net: Semantic segmentation with camera-LiDAR cross-attention fusion based on fast neighbor feature aggregation," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 19627–19639, 2024.

[3] J. Wan, Z. Zeng, Q. Qiu, Z. Xie, and Y. Xu, "PointNest: Learning deep multiscale nested feature propagation for semantic segmentation of 3-D point clouds," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 16, pp. 9051–9066, 2023.

[4] X. Li, Z. Zhang, Y. Li, M. Huang, and J. Zhang, "SFL-NET: Slight filter learning network for point cloud semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5703914.

[5] J. Zhao et al., "SemanticFlow: Semantic segmentation of sequential LiDAR point clouds from sparse frame annotations," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5701611.

[6] D. Chen, Y. Wang, L. Zhang, and Z. Kang, "Enhanced local feature learning with simple offset attention for semantic segmentation of large-scale point clouds," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5705916.

[7] Q. Zhang, Y. Peng, Z. Zhang, and T. Li, "Semantic segmentation of spectral LiDAR point clouds based on neural architecture search," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5403811.

[8] R. Zhang, G. Zhang, J. Yin, X. Jia, and A. Mian, "Mesh-based DGCNN: Semantic segmentation of textured 3-D urban scenes," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4402812.

[9] C. Liu et al., "Context-aware network for semantic segmentation toward large-scale point clouds in urban environments," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5703915.

[10] Y. Pan, B. Gao, J. Mei, S. Geng, C. Li, and H. Zhao, "SemanticPOSS: A point cloud dataset with large quantity of dynamic instances," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Oct. 2020, pp. 687–693.

[11] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2012, pp. 3354–3361.

[12] B. Fu et al., "Quantifying vegetation species functional traits along hydrologic gradients in Karst wetland based on 3D mapping with UAV hyperspectral point cloud," *Remote Sens. Environ.*, vol. 307, Jun. 2024, Art. no. 114160.

[13] L. Deng et al., "Comparison of 2D and 3D vegetation species mapping in three natural scenarios using UAV-LiDAR point clouds and improved deep learning methods," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 125, Dec. 2023, Art. no. 103588.

[14] B. Wu, A. Wan, X. Yue, and K. Keutzer, "SqueezeSeg: Convolutional neural nets with recurrent CRF for real-time road-object segmentation from 3D LiDAR point cloud," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 1887–1893.

[15] B. Wu, X. Zhou, S. Zhao, X. Yue, and K. Keutzer, "SqueezeSegV2: Improved model structure and unsupervised domain adaptation for road-object segmentation from a LiDAR point cloud," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2019, pp. 4376–4382.

[16] A. Milioto, I. Vizzo, J. Behley, and C. Stachniss, "RangeNet++: Fast and accurate LiDAR semantic segmentation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 4213–4220.

[17] S. Li, X. Chen, Y. Liu, D. Dai, C. Stachniss, and J. Gall, "Multi-scale interaction for real-time LiDAR data segmentation on an embedded platform," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 738–745, Apr. 2022.

[18] Y. Zhao, L. Bai, and X. Huang, "FIDNet: LiDAR point cloud semantic segmentation with fully interpolation decoding," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2021, pp. 4453–4458.

[19] H. Cheng, X. Han, and G. Xiao, "CENet: Toward concise and efficient LiDAR semantic segmentation for autonomous driving," in *Proc. IEEE Int. Conf. Multimedia Expo (ICME)*, Jul. 2022, pp. 1–6.

[20] H.-X. Cheng, X.-F. Han, and G.-Q. Xiao, "TransRVNet: LiDAR semantic segmentation with transformer," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 5895–5907, Jun. 2023.

[21] W. Huang et al., "OPOCA: One point one class annotation for LiDAR point cloud semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5701610.

[22] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 652–660.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.

[23] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," 2017, *arXiv:1706.02413*.

[24] M. Tatarchenko, J. Park, V. Koltun, and Q.-Y. Zhou, "Tangent convolutions for dense prediction in 3D," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3887–3896.

[25] R. A. Rosu, P. Schütt, J. Quenzel, and S. Behnke, "LatticeNet: Fast point cloud segmentation using permutohedral lattices," 2019, *arXiv:1912.05905*.

[26] Q. Hu et al., "RandLA-Net: Efficient semantic segmentation of large-scale point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11108–11117.

[27] H. Shuai, X. Xu, and Q. Liu, "Backward attentive fusing network with local aggregation classifier for 3D point cloud semantic segmentation," *IEEE Trans. Image Process.*, vol. 30, pp. 4973–4984, 2021.

[28] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas, "KPConv: Flexible and deformable convolution for point clouds," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6411–6420.

[29] F. Zhang, J. Fang, B. W. Wah, and P. H. S. Torr, "Deep fusionnet for point cloud semantic segmentation," in *Proc. Eur. Conf. Comput. Vis.*, Aug. 2020, pp. 644–663.

[30] K. Liu, Z. Gao, F. Lin, and B. M. Chen, "FG-Net: A fast and accurate framework for large-scale LiDAR point cloud understanding," *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 553–564, Jan. 2023.

[31] C. Choy, J. Gwak, and S. Savarese, "4D spatio-temporal ConvNets: Minkowski convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3070–3079.

[32] S. Qiu, S. Anwar, and N. Barnes, "Semantic segmentation for real point cloud scenes via bilateral augmentation and adaptive fusion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 1757–1767.

[33] H. Tang et al., "Searching efficient 3D architectures with sparse pointvoxel convolution," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 685–702.

[34] H. Zhou et al., "Cylinder3D: An effective 3D framework for driving-scene LiDAR semantic segmentation," 2020, *arXiv:2008.01550*.

[35] X. Yan et al., "Sparse single sweep LiDAR point cloud segmentation via learning contextual shape priors from scene completion," 2020, *arXiv:2012.03762*.

[36] R. Cheng, R. Razani, E. Taghavi, E. Li, and B. Liu, "$(AF)^2$-S3Net: Attentive feature fusion with adaptive feature selection for sparse semantic segmentation network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 12547–12556.

[37] J. Xu, R. Zhang, J. Dou, Y. Zhu, J. Sun, and S. Pu, "RPVNet: A deep and efficient range-point-voxel fusion network for LiDAR point cloud segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 16004–16013.

[38] C. Xu et al., "Squeezesegv3: Spatially-adaptive convolution for efficient point-cloud segmentation," in *Proc. 16th Eur. Conf. Comput. Vis. (ECCV)*. Glasgow, U.K.: Springer, Aug. 2020, pp. 1–19.

[39] I. Alonso, L. Riazuelo, L. Montesano, and A. C. Murillo, "3D-MiniNet: Learning a 2D representation from point clouds for fast and efficient 3D LiDAR semantic segmentation," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 5432–5439, Oct. 2020.

[40] A. Xiao, X. Yang, S. Lu, D. Guan, and J. Huang, "FPS-Net: A convolutional fusion network for large-scale LiDAR point cloud segmentation," *ISPRS J. Photogramm. Remote Sens.*, vol. 176, pp. 237–249, Jun. 2021.

[41] Y. Zhang et al., "PolarNet: An improved grid representation for online LiDAR point clouds semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9601–9610.

[42] S. Wen, T. Wang, and S. Tao, "Hybrid CNN-LSTM architecture for LiDAR point clouds semantic segmentation," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 5811–5818, Jul. 2022.

[43] M. Gerdzhev, R. Razani, E. Taghavi, and L. Bingbing, "TORNADO-Net: Multiview total variation semantic segmentation with diamond inception module," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Xi'an, China, May 2021, pp. 9543–9549.

[44] D. Kochanov, F. Karimi Nejadasl, and O. Booij, "KPRNet: Improving projection-based LiDAR semantic segmentation," 2020, *arXiv:2007.12668*.

[45] Y. Ali Alnaggar, M. Afifi, K. Amer, and M. ElHelw, "Multi projection fusion for real-time semantic segmentation of 3D LiDAR point clouds," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 1799–1808.

[46] T. Cortinhal, G. Tzelepis, and E. E. Aksoy, "SalsaNext: Fast, uncertaintyaware semantic segmentation of LiDAR point clouds," in *Proc. Int. Symp. Vis. Comput.* Cham, Switzerland: Springer, 2020, pp. 207–222.

[47] X. Ma, C. Qin, H. You, H. Ran, and Y. Fu, "Rethinking network design and local geometry in point cloud: A simple residual MLP framework," in *Proc. ICLR*, 2022, p. 5019.

[48] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph CNN for learning on point clouds," *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, Oct. 2019.

[49] A. X. Chang et al., "ShapeNet: An information-rich 3D model repository," 2015, *arXiv:1512.03012*.

[50] H. Qiu, B. Yu, and D. Tao, "GFNet: Geometric flow network for 3D point cloud semantic segmentation," *Trans. Mach. Learn. Res.*, Sep. 2022. [Online]. Available: https://openreview.net/forum?id=LSAAlS7Yts

[51] X. Wu, Y. Lao, L. Jiang, X. Liu, and H. Zhao, "Point transformer v2: Grouped vector attention and partition-based pooling," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 33330–33342.

[52] X. Wu et al., "Point transformer v3: Simpler, faster, stronger," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2024, pp. 4840–4851.

[53] B. Peng et al., "OA-CNNs: Omni-adaptive sparse CNNs for 3D semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2024, pp. 21305–21315.

[54] Y. Xu et al., "NeiEA-NET: Semantic segmentation of large-scale point cloud scene via neighbor enhancement and aggregation," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 119, May 2023, Art. no. 103285.

[55] S. Shi et al., "PV-RCNN: Point-voxel feature set abstraction for 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 10526–10535.

[56] X. Xu, L. Kong, H. Shuai, and Q. Liu, "FRNet: Frustum-range networks for scalable LiDAR segmentation," *IEEE Trans. Image Process.*, vol. 34, pp. 2173–2186, 2025.

[57] A. Ando, S. Gidaris, A. Bursuc, G. Puy, A. Boulch, and R. Marlet, "RangeViT: Towards vision transformers for 3D semantic segmentation in autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 5240–5250.

[58] L. Kong et al., "Rethinking range view representation for LiDAR segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Paris, France, Oct. 2023, pp. 228–240.

[59] X. Yan, C. Zheng, Z. Li, S. Wang, and S. Cui, "PointASNL: Robust point clouds processing using nonlocal neural networks with adaptive sampling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 5588–5597.

[60] R. Razani, R. Cheng, E. Taghavi, and L. Bingbing, "Lite-HDSeg: LiDAR semantic segmentation using lite harmonic dense convolutions," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 9550–9556.

[61] R. Li, S. Li, X. Chen, T. Ma, J. Gall, and J. Liang, "TFNet: Exploiting temporal cues for fast and accurate LiDAR semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 4547–4556.

[62] B. Ding, "LENet: Lightweight and efficient LiDAR semantic segmentation using multi-scale convolution attention," 2023, *arXiv:2301.04275*.

[63] F. Liu, X. Chen, Z. Huang, and Z. Liu, "LiDAR point cloud semantic segmentation method based on multi-scale contextual feature," in *Proc. 35th Chin. Control Decis. Conf. (CCDC)*, Yichang, China, May 2023, pp. 1–6.

[64] M. Berman, A. R. Triki, and M. B. Blaschko, "The Lovasz-softmax loss: A tractable surrogate for the optimization of the intersection-over-union measure in neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4413–4421.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:29:30 UTC from IEEE Xplore. Restrictions apply.